

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA0702

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards
(BL2, BL6)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:25

Annexure A

Coding challenge

Code of Conduct:

I Abarajithan S(192521299) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Abarajithan S

Annexure A

Coding challenge

1. Write a Python script that generates a star topology diagram (ASCII or graphical) given n hosts and a central switch. The script must print each host label, the switch name, and the connecting Cat6 UTP cable length.
2. Code a function `validate\_segment(length\_m)` that returns True if the UTP segment is within IEEE 802.3 limits (≤ 100 m) and False with an error message otherwise. Include at least 3 test cases.

3. Write a script that simulates CSMA/CD on a star topology with 4 hosts. Log every collision detection event and the back-off interval to a file named 'csma_log.txt'.
4. Using Python's 'socket' library, write a client-server program that models two hosts communicating through a star switch. Demonstrate message forwarding by printing the switch's MAC address table after each frame transmission.
5. Create a class 'StarSwitch' with methods 'add_port(host_id)', 'remove_port(host_id)', and 'broadcast(frame)'. Broadcast must send a frame to all ports except the source and log each delivery.

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done

Program 1: Star Topology Diagram

```
def create_star_topology(n):
    switch_name = "SWITCH-01"

    print("\nSTAR TOPOLOGY")
    print("=" * 40)
    print("Central Device:", switch_name)
    print("Cable Type: Cat6 UTP")
    print()

    for i in range(1, n + 1):
        length = 10 + (i * 5)

        print("Host-{} ----- {} m ----- {}".format(
            i, length, switch_name
        ))

    print("=" * 40)

try:
    n = int(input("Enter number of hosts: "))

    if n <= 0:
        print("Error: Number of hosts must be greater than 0.")
    else:
        create_star_topology(n)

except ValueError:
    print("Error: Please enter a valid integer.")
```

Output:

Enter number of hosts: 4

STAR TOPOLOGY

Central Device: SWITCH-01
Cable Type: Cat6 UTP

Host-1 ----- 15 m ----- SWITCH-01
Host-2 ----- 20 m ----- SWITCH-01
Host-3 ----- 25 m ----- SWITCH-01
Host-4 ----- 30 m ----- SWITCH-01

Program 2: UTP Segment Length Validation

```
def validate_segment(length_m):  
    if length_m <= 100:  
        return True  
    else:  
        print("Error: UTP segment length cannot exceed 100 meters.")  
        return False  
  
# Test Case 1  
print("Test Case 1:")  
print("Length = 50 m")  
print("Result:", validate_segment(50))  
  
# Test Case 2  
print("\nTest Case 2:")  
print("Length = 100 m")  
print("Result:", validate_segment(100))  
  
# Test Case 3  
print("\nTest Case 3:")  
print("Length = 120 m")  
print("Result:", validate_segment(120))
```

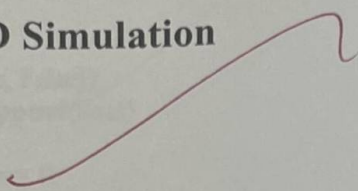
Output:

Test Case 1:
Length = 50 m
Result: True

Test Case 2:
Length = 100 m
Result: True

Test Case 3:
Length = 120 m
Error: UTP segment length cannot exceed 100 meters.
Result: False

Program 3: CSMA/CD Simulation



```

import random
import time
LOG_FILE = "csma_log.txt"
def log_message(message):
    with open(LOG_FILE, "a") as file:
        file.write(message + "\n")
def csma_cd_simulation():
    hosts = ["Host-1", "Host-2", "Host-3", "Host-4"]
    # Clear old log
    with open(LOG_FILE, "w") as file:
        file.write("CSMA/CD SIMULATION LOG\n")
        file.write("=" * 40 + "\n")
    print("CSMA/CD Simulation Started")
    print("=" * 40)
    for round_no in range(1, 6):
        print("\nRound", round_no)
        # Randomly select transmitting hosts
        transmitting_hosts = []
        for host in hosts:
            if random.choice([True, False]):
                transmitting_hosts.append(host)
        # No host transmitted
        if len(transmitting_hosts) == 0:
            message = "No host transmitted in this round."
            print(message)
            log_message(message)
        # Only one host transmitted
        elif len(transmitting_hosts) == 1:
            host = transmitting_hosts[0]
            message = host + " transmitted successfully."
            print(message)
            log_message(message)
        # More than one host transmitted
        else:
            message = "COLLISION DETECTED: " + ", ".join(transmitting_hosts)
            print(message)
            log_message(message)
            for host in transmitting_hosts:
                backoff = random.randint(1, 10)
                message = host + " back-off interval: " + str(backoff) + " ms"
                print(message)
                log_message(message)
    print("\nSimulation completed.")
    print("Collision and back-off details saved in", LOG_FILE)
csma_cd_simulation()

```

Output:


```

CSMA/CD Simulation Started
=====

Round 1
Host-4 transmitted successfully.

Round 2
COLLISION DETECTED: Host-1, Host-2
Host-1 back-off interval: 8 ms
Host-2 back-off interval: 5 ms

Round 3
Host-2 transmitted successfully.

Round 4
COLLISION DETECTED: Host-1, Host-2
Host-1 back-off interval: 6 ms
Host-2 back-off interval: 2 ms

Round 5
COLLISION DETECTED: Host-2, Host-3
Host-2 back-off interval: 9 ms
Host-3 back-off interval: 1 ms

Simulation completed.
Collision and back-off details saved in csma_log.txt

```

Program 4A: Star Switch Server

```

import socket
HOST = "127.0.0.1"
PORT = 5000
mac_table = {
    "AA:AA:AA:AA:AA:01": "Host-1",
    "BB:BB:BB:BB:BB:02": "Host-2"
}
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen(1)
print("Star Switch is running...")
print("Waiting for Host-1 connection...")
connection, address = server.accept()
print("Host connected:", address)
while True:
    data = connection.recv(1024).decode()
    if not data:
        break
    print("\nFrame received:", data)
    print("Switch MAC Address Table:")
    print("-" * 35)
    for mac, host in mac_table.items():
        print(mac, "->", host)
    print("-" * 35)
    if data.lower() == "exit":
        break
    response = "Frame forwarded through Star Switch"
    connection.send(response.encode())
connection.close()
server.close()
print("Server closed.")

```

Client Output

Connected to Star Switch.

Host-1 Enter message: Hello Host-2

Switch: Frame forwarded through Star Switch

Host-1 Enter message: Computer Networks

Switch: Frame forwarded through Star Switch

Host-1 Enter message: exit

Connection closed.

Program 4B: Host Client

```
import socket
```

```
HOST = "127.0.0.1"
```

```
PORT = 5000
```

```
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
client.connect((HOST, PORT))
```

```
print("Connected to Star Switch.")
```

```
while True:
```

```
    message = input("Host-1 Enter message: ")
```

```
    client.send(message.encode())
```

```
    if message.lower() == "exit":
```

```
        break
```

```
    response = client.recv(1024).decode()
```

```
    print("Switch:", response)
```

```
client.close()
```

```
print("Connection closed.")
```

Server Output

Star Switch is running...
Waiting for Host-1 connection...

Frame received: Hello Host-2

Switch MAC Address Table:

AA:AA:AA:AA:AA:01 -> Host-1
BB:BB:BB:BB:BB:02 -> Host-2

Frame received: Computer Networks

Switch MAC Address Table:

AA:AA:AA:AA:AA:01 -> Host-1
BB:BB:BB:BB:BB:02 -> Host-2

Program 5: StarSwitch Class

class StarSwitch:

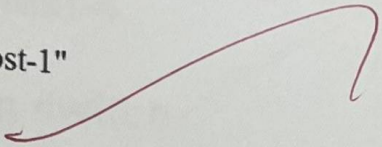
```
def __init__(self):  
    self.ports = []  
  
def add_port(self, host_id):  
    if host_id not in self.ports:  
        self.ports.append(host_id)  
        print(host_id, "connected to switch.")  
    else:  
        print(host_id, "is already connected.")  
  
def remove_port(self, host_id):  
    if host_id in self.ports:  
        self.ports.remove(host_id)  
        print(host_id, "removed from switch.")  
    else:  
        print("Error:", host_id, "is not connected.")  
  
def broadcast(self, frame):  
    source = frame["source"]  
    message = frame["message"]  
  
    print("\nBroadcasting frame...")  
    print("Source:", source)  
    print("Message:", message)  
  
    if source not in self.ports:  
        print("Error: Source host is not connected.")  
    return
```

```
for port in self.ports:  
    if port != source:  
        print("Frame delivered to", port)
```

```
# Create switch  
switch = StarSwitch()
```

```
# Add hosts  
switch.add_port("Host-1")  
switch.add_port("Host-2")  
switch.add_port("Host-3")  
switch.add_port("Host-4")
```

```
# Broadcast frame  
frame = {  
    "source": "Host-1",  
    "message": "Hello from Host-1"  
}
```



```
switch.broadcast(frame)
```

```
# Remove a host  
print()  
switch.remove_port("Host-4")
```

```
# Broadcast again  
frame2 = {  
    "source": "Host-2",  
    "message": "Network message"  
}
```

```
switch.broadcast(frame2)
```


Sample Output:

**Host-1 connected to switch.
Host-2 connected to switch.
Host-3 connected to switch.
Host-4 connected to switch.**

Broadcasting frame...

Source: Host-1

Message: Hello from Host-1

Frame delivered to Host-2

Frame delivered to Host-3

Frame delivered to Host-4

Host-4 removed from switch.

Broadcasting frame...

Source: Host-2

Message: Network message

Frame delivered to Host-1

Frame delivered to Host-3

Q.2.2